SUCHEN: BINÄRE SUCHE

O(LOG N)

```
int binSearch(int[] a, int val) {
  int l = -1, h = a.length;
  while (l + 1 != h) {
    int m = l + (h - l) / 2;
    if (a[m] < val) l = m;
    else h = m;
  }
  return h; // Index des Elements od. Einfügepunkt
}
```

- **Halboffen [left, right):** Start `right = list.size()`; `while (left < right)`.
- **Geschlossen [left, right]:** Start `right = list.size() - 1`; `while (left <= right)`.
- **Falle:** `right = list.size()` mit `while (left <= right)` erzeugt `IndexOutOfBoundsException` bei `m = list.size()`!

SORTIEREN: INSERTION SORT

O(N²)

```
void insertionSort(int[] a) {
  for (int i = 1; i < a.length; ++i) {
    int key = a[i], j = i - 1;
    while (j >= 0 && a[j] > key) {
      a[j + 1] = a[j];
      j--;
    }
    a[j + 1] = key;
  }
}
```

Fügt Element `i` an passender Stelle in bereits sortierten linken Teilbereich ein. Best Case: O(n) bei fast sortiert.

SORTIEREN: SELECTION SORT

O(N²)

```
void selectionSort(int[] a) {
  for (int i = 0; i < a.length - 1; i++) {
    int min = i;
    for (int j = i + 1; j < a.length; j++) {
      if (a[j] < a[min]) min = j;
    }
    int temp = a[min]; a[min] = a[i]; a[i] = temp;
  }
}
```

Sucht kleinstes Element im unsortierten Teil und tauscht es nach vorne. Immer O(n²) Vergleiche.

JAVA GRUNDLAGEN & OVERLOADING

```
class MyClass {
  int x; // Attribut
  MyClass(int value) { x = value; } // Konstruktor
  public int getX() { return x; } // Getter
}

class Calculator {
  int add(int a, int b) { return a + b; }
  // Overloading: Gleicher Name, andere Parameter
  int add(int a, int b, int c) { return a + b + c; }
  double add(double a, double b) { return a + b; }
}
```

Overloading (Überladen): Unterscheidung nur durch Parametertypen/-anzahl (Rückgabetyp reicht nicht!).

ALIASING & DEFENSIVE COPYING

Aliasing: Objektzuweisungen & Parameterübergaben kopieren nur die **Referenz**, nie das Objekt selbst!

Änderungen wirken auf das Original. Schutz durch *Defensive Copying* (neues Objekt beim Empfangen/Rückgeben) oder *Immutability*.

IMMUTABILITY (UNVERÄNDERLICHKEIT)

- Attribute stets `private final` dekarieren.
- **Keine Setter/Mutatoren** bereitstellen.
- Arithmetische/Verändernde Methoden (wie `plus`, `add`) verändern **nie `this`**, sondern liefern immer ein **neues Objekt** zurück:

```
public Amount plus(Amount other) {
  // NIEMALS this.galleons += ...
  return new Amount(this.galleons + other.galleons, ...);
}
```

PRECONDITIONS & EXCEPTIONS

Vorbedingungen sofort zu Beginn prüfen:

```
if (val < 0)
  throw new IllegalArgumentException("Negative: " + val);
```

ASYMPTOTISCHE KOMPLEXITÄT (O-NOTATION)

Klasse	Bezeichnung	Beispiel	Zuwachs f(2n)/f(n)
O(1)	Konstant	Array-Zugriff / Stack pop	1
O(log n)	Logarithmisch	Binäre Suche	1 + log(2)/log(n)
O(n)	Linear	Sequenzielle Suche	2
O(n log n)	Log-Linear	Mergesort / Quicksort (Avg)	2 + (2 log 2 / log n)
$O(n^2)$	Quadratisch	Selection / Insertion Sort	4
$O(n^3)$	Kubisch	Naive Matrix-Multiplikation	8
$O(2^n)$	Exponentiell	Brute-Force Knapsack	2^n
$O(3^n)$	Exponentiell	Brute Force Coloring Problem (3 Colors)	3^n
O(n!)	Fakultät	Naive Traveling Salesman	n!

RECHENREGELN O-NOTATION

1. **Konstanten streichen:** $15 * n^3 \in O(n^3)$, $n^3/1000 \in O(n^3)$
2. **Höchster Term dominiert:** $n^2 + 50n + 10^6 \in O(n^2)$
3. **Polynomgrad entscheidet:** $15n^5 + 20n^3 \in O(n^5)$
4. **Log-Basis egal:** $\log_2 n, \log_{10} n \in O(\log n)$
5. **Exponenten-Basis zählt:** $2^n \notin O(3^n)$, $3^n \in O(3^n)$
6. **Exponent wächst schneller als jedes Polynom:** $n^{100} \in O(2^n)$
7. **Produkte bleiben bestehen:** $n * \log(n) \in O(n \log n)$

TYPISCHE SCHLEIFENMUSTER

- `for (int i=0; i*i < n; ++i)` → **O(√n)**
- `for (int j=n; j > 0; j /= 2)` → **O(log n)**
- `for (int i=0; i < 1000; ++i)` → **O(1)** (Konstante Grenzen ändern O-Klasse nicht!)

ARRAY VS. ARRAYLIST

Aktion	Array (`Lamp[]`)	ArrayList (`List<Lamp>`)
Erzeugen	`new Lamp[n]`	`new ArrayList<>(n)`
Lesen	`arr[i]`	`lst.get(i)`
Schreiben	`arr[i] = l`	`lst.set(i, l)`
Größe	`arr.length`	`lst.size()`

JAVA COLLECTIONS FRAMEWORK

ArrayList: Erlaubt Duplikate, behält Reihenfolge bei. `remove("x")` löscht 1. Vorkommen (→ `boolean`). Ausgabe: `[a, b]` .

HashSet: Keine Duplikate! `add()` auf Vorhandenes wird ignoriert. Keine Reihenfolge. `remove("fehlt")` → **false** (keine Exception!).

HashMap<K,V>: Key-Value-Paare (Keys eindeutig). `put(k, v)` überschreibt alten Wert. `keySet()` = Keys, `values()` = Werte.

- `map.get(key)` liefert `null` wenn Key fehlt (keine Exception!).
- **Map.of(...)** erzeugt eine **unveränderliche (immutable)** Map!

PRÜFUNGSFALLEN: GLEICHHEIT & NULL

- **Referenzvergleich (`==`):** Vergleicht Speicheradressen. `null == null` ergibt **true**!
- **Inhaltsvergleich (`.equals()`):** Wenn `equals()` in eigener Klasse nicht überschrieben ist, macht es wie `==` nur Referenzvergleich!
- **String-Vergleich mit `==` :** Vergleicht Referenzen, nicht den Text! `"Hallo" == "Hallo"` ist durch den String-Pool zufällig `true` , aber `strA == strB` bei Laufzeit-Strings ist `false` . **Immer `.equals()` nutzen!**
- **NullPointerException:** Aufruf von Methoden auf `null` bricht Programm ab (z. B. `null.getTitle()` → Absturz!).